

XS WALLET

Guia de Implementação para Desenvolvedores

Plano de 10 Semanas - 6 Fases

Documento	Guia de Implementação
Versão	1.0.0
Baseado em	Especificação Técnica v0.2.0
Data	Janeiro 2026
Público-Alvo	Desenvolvedores, Implementadores
Duração Estimada	10 semanas

1. Visão Geral do Projeto

1.1 Objetivo

Implementar uma carteira desktop self-custody com atomic swaps entre BTC, Liquid e Lightning Network, seguindo os princípios de **Zero Trust** e **Crash Recovery**.

1.2 Arquitetura Alvo

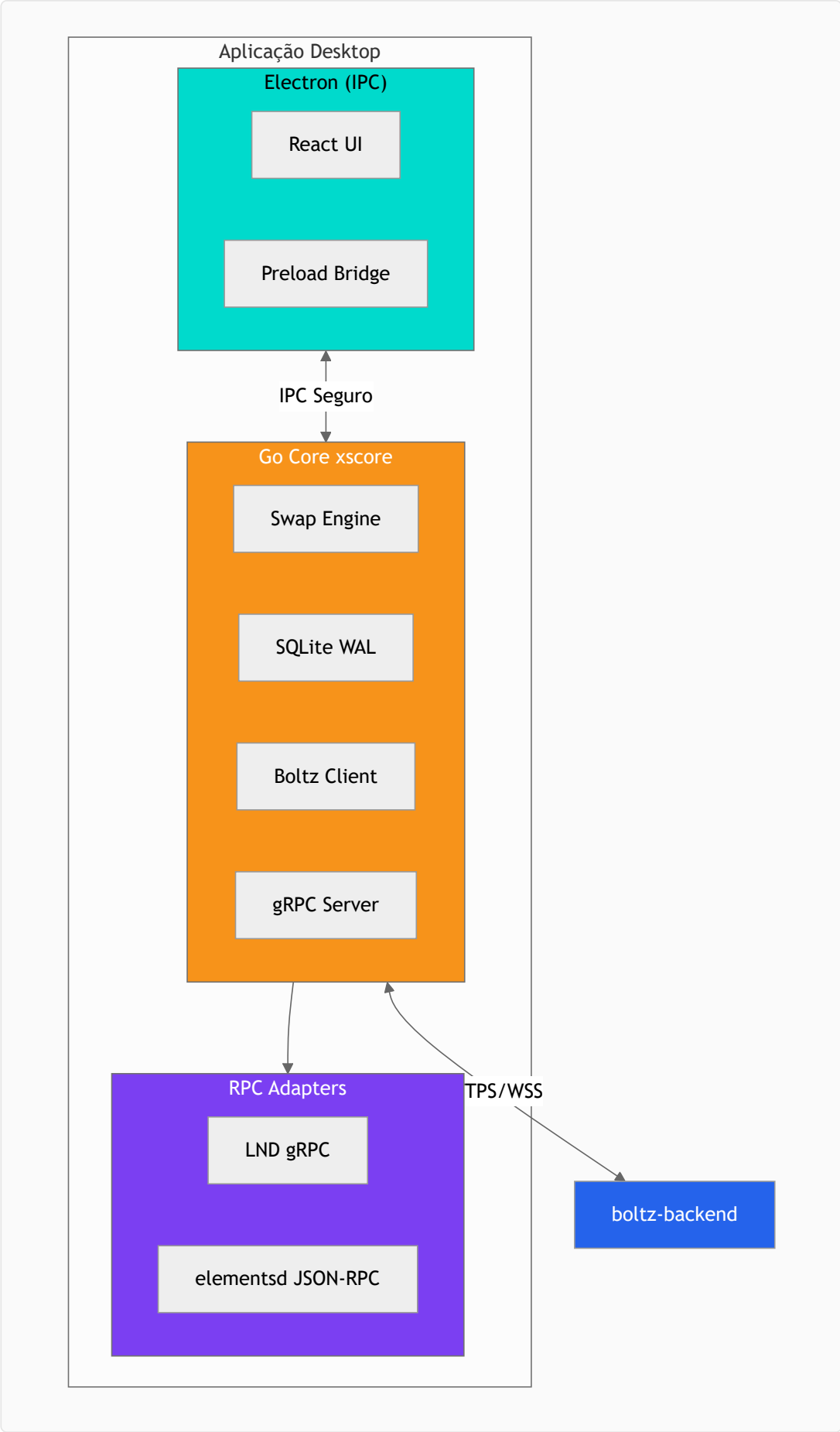


Figura 1.1: Arquitetura Alvo

1.3 Estado Atual vs Alvo

Componente	Estado Atual	Alvo
------------	--------------	------

Frontend	React + Vite (HTTP via api-bridge)	Electron + IPC
Go Core	✓ Implementado (parcial)	Completo com todos orchestrators
Swaps	Submarine (parcial)	Submarine + Reverse + Chain
MuSig2	Não implementado	Claim/refund cooperativo
Adapters	Apenas bitcoind RPC	LND gRPC + elementsd JSON-RPC

Tabela 1.1: Gap Analysis

2. Roadmap de Implementação

2.1 Visão Geral das Fases

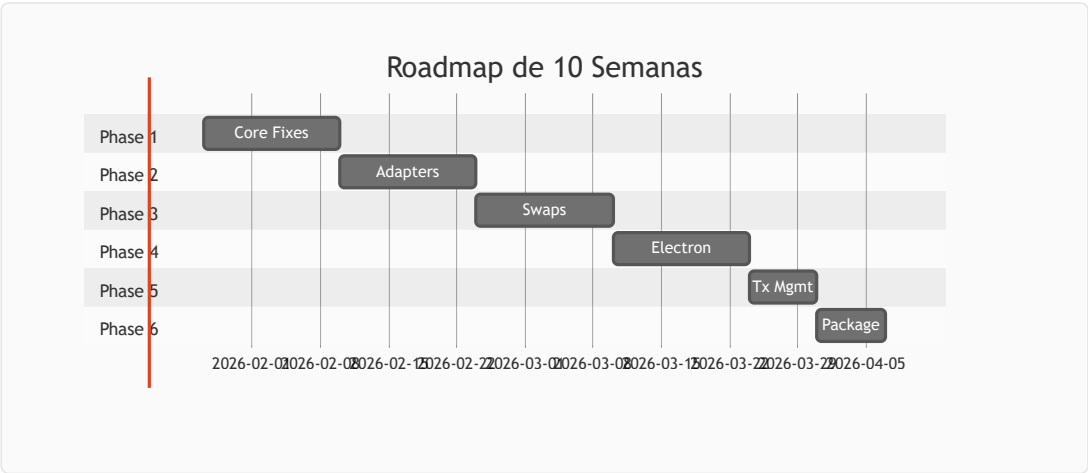


Figura 2.1: Timeline do Projeto

2.2 Dependency Graph

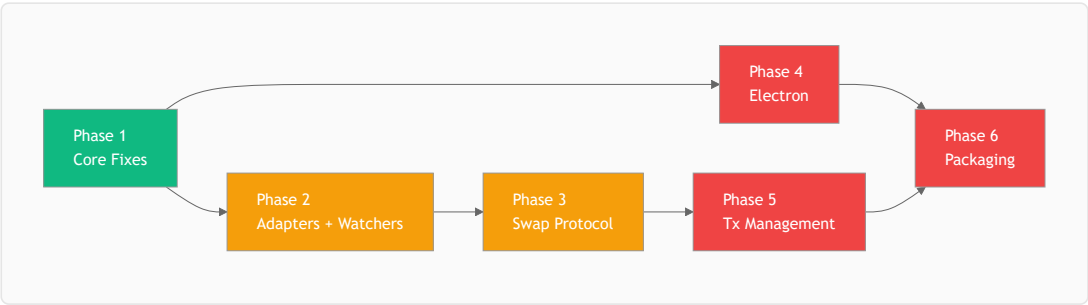


Figura 2.2: Dependências entre Fases

3. Phase 1: Core Fixes (Semanas 1-2)

✓ **Objetivo:** Consolidar a base de dados e corrigir bugs críticos antes de adicionar funcionalidades.

3.1 QuoteSwap Kind/Chain Mapping

Problema

Arquivo `core/internal/server/swap_service.go` (linhas 42-45) `hardcodea BTC→LN` e usa `req.Kind.String()` que não casa com `provider.SwapKind`.

Solução

```
// Criar helpers de conversão
func protoKindToProvider(k pb.SwapKind) provider.SwapKind {
    switch k {
        case pb.SwapKind_SWAP_KIND_SUBMARINE:
            return provider.SwapKindSubmarine
        case pb.SwapKind_SWAP_KIND_REVERSE:
            return provider.SwapKindReverse
        case pb.SwapKind_SWAP_KIND_CHAIN:
            return provider.SwapKindChain
        default:
            return ""
    }
}

func protoChainToProvider(from, to pb.Chain) (provider.Chain, provider.Chain) {
    // Mapear pb.Chain_CHAIN_BTC → provider.ChainBTC, etc
}
```

Arquivos Afetados

- `core/internal/server/swap_service.go` - adicionar helpers
- `core/internal/server/swap_service.go` - atualizar `QuoteSwap()`

3.2 Schema Unificado

Problema

Existem dois schemas divergentes:

- `core/internal/db/db.go` (~35 colunas)
- `database/schema.sqlite.sql` (40+ colunas)

Solução

Consolidar em `core/internal/db/db.go` adicionando colunas faltantes:

```
-- Adicionar ao schema em db.go
provider TEXT NOT NULL DEFAULT 'boltz',
provider_url TEXT NOT NULL,
from_asset TEXT NOT NULL,
to_asset TEXT NOT NULL,
boltz_status TEXT,
boltz_raw TEXT,
swap_tree_hex TEXT,
tap_tweak32_hex TEXT,
encrypted_preimage BLOB, -- NOVO: substituir preimage_hex
lockup_seen_mempool_at TEXT,
lockup_confirmed_height INTEGER,
claim_chain TEXT,
claim_seen_mempool_at TEXT,
claim_confirmed_height INTEGER,
refund_chain TEXT,
refund_seen_mempool_at TEXT,
refund_confirmed_height INTEGER
```

Migração

```
-- migrations/001_add_spec_columns.sql
ALTER TABLE swaps ADD COLUMN encrypted_preimage BLOB;
ALTER TABLE swaps ADD COLUMN from_asset TEXT;
ALTER TABLE swaps ADD COLUMN to_asset TEXT;
-- ... etc

-- Criar tabela vault_lockout
CREATE TABLE vault_lockout (
  id INTEGER PRIMARY KEY CHECK(id = 1),
  attempts INTEGER DEFAULT 0,
  last_attempt_at TEXT,
  locked_until TEXT
);
```

Arquivos Afetados

- `core/internal/db/db.go` - atualizar schema
- `database/schema.sqlite.sql` - REMOVER
- `migrations/001_add_spec_columns.sql` - CRIAR

3.3 Preimage Criptografada

⚠ CRÍTICO: A preimage R atualmente é salva em plaintext no DB. Isso viola o spec de segurança.

Implementação

Arquivo: `core/internal/vault/vault.go`

```
// Adicionar métodos ao Vault
func (v *Vault) EncryptPreimage(preimage []byte) ([]byte, error) {
  v.mu.RLock()
  defer v.mu.RUnlock()

  if !v.unlocked || v.seed == nil {
    return nil, ErrVaultLocked
  }

  // Derivar chave de criptografia do seed
  key := derivePreimageKey(v.seed)

  // Criptografar com AES-256-GCM
  nonce, ciphertext, err := encrypt(preimage, key[:])
  if err != nil {
    return nil, err
  }

  // Retornar nonce || ciphertext
  return append(nonce, ciphertext...), nil
}

func (v *Vault) DecryptPreimage(encrypted []byte) ([]byte, error) {
  v.mu.RLock()
  defer v.mu.RUnlock()

  if !v.unlocked || v.seed == nil {
    return nil, ErrVaultLocked
  }

  // Separar nonce e ciphertext
  nonce := encrypted[:nonceLength]
  ciphertext := encrypted[nonceLength:]

  key := derivePreimageKey(v.seed)
  return decrypt(ciphertext, nonce, key[:])
}
```

Arquivo: `core/internal/swap/engine.go`

```
// Alterar Create() para usar encrypted_preimage
func (e *Engine) Create(...) (*Swap, error) {
  // ... existing code ...
}
```

```

// Criptografar preimage
encryptedPreimage, err := vault.EncryptPreimage(preimage)
if err != nil {
    return nil, err
}

// Salvar encrypted_preimage em vez de preimage_hex
_, err = e.db.ExecContext(ctx, `
    INSERT INTO swaps (id, ..., encrypted_preimage, preimage_hash_hex, ...)
    VALUES (?, ..., ?, ?, ...)
`, id, ..., encryptedPreimage, hex.EncodeToString(hash), ...)
}

```

Arquivos Afetados

- `core/internal/vault/vault.go` - adicionar `EncryptPreimage/DecryptPreimage`
- `core/internal/swap/engine.go` - usar `encrypted_preimage`
- `core/internal/db/db.go` - schema com `encrypted_preimage`

3.4 PIN Lockout Persistente

Implementação

Arquivo: `core/internal/vault/lockout.go`

```

package vault

import (
    "database/sql"
    "time"
    "github.com/xs-wallet/xscore/internal/db"
)

type LockoutManager struct {
    db          *db.DB
    maxAttempts int          // 10
    lockoutTime time.Duration // 30 min
}

func NewLockoutManager(database *db.DB) *LockoutManager {
    return &LockoutManager{
        db:          database,
        maxAttempts: 10,
        lockoutTime: 30 * time.Minute,
    }
}

func (l *LockoutManager) RecordAttempt(success bool) error {
    if success {
        // Reset attempts
        _, err := l.db.Exec(`
            UPDATE vault_lockout SET attempts = 0, locked_until = NULL
            WHERE id = 1
        `)
        return err
    }

    // Increment attempts
    _, err := l.db.Exec(`
        INSERT INTO vault_lockout (id, attempts, last_attempt_at)
        VALUES (1, 1, datetime('now'))
        ON CONFLICT(id) DO UPDATE SET
            attempts = attempts + 1,
            last_attempt_at = datetime('now'),
            locked_until = CASE
                WHEN attempts + 1 >= ? THEN datetime('now', '+30 minutes')
                ELSE locked_until
            END
    `, l.maxAttempts)
    return err
}

func (l *LockoutManager) CanAttempt() (bool, time.Duration, error) {
    var attempts int
    var lockedUntil sql.NullString

```

```
err := l.db.QueryRow(`
    SELECT attempts, locked_until FROM vault_lockout WHERE id = 1
`).Scan(&attempts, &lockedUntil)

if err == sql.ErrNoRows {
    return true, 0, nil
}
if err != nil {
    return false, 0, err
}

if lockedUntil.Valid {
    until, _ := time.Parse("2006-01-02 15:04:05", lockedUntil.String)
    if time.Now().Before(until) {
        return false, time.Until(until), nil
    }
}

return true, 0, nil
}
```

Integração no Vault:

```
// Em vault.go
func (v *Vault) Unlock(pin string) (string, error) {
    // Verificar lockout
    canAttempt, waitTime, err := v.lockout.CanAttempt()
    if err != nil {
        return "", err
    }
    if !canAttempt {
        return "", fmt.Errorf("locked out for %v", waitTime)
    }

    // ... existing unlock logic ...

    // Registrar resultado
    v.lockout.RecordAttempt(err == nil)

    return sessionID, err
}
```

Arquivos Afetados

- core/internal/vault/lockout.go - CRIAR
- core/internal/vault/vault.go - integrar LockoutManager
- core/internal/db/db.go - adicionar tabela vault_lockout

3.5 Checklist Phase 1

Tarefa	Arquivos	Status
QuoteSwap fix	swap_service.go	Pendente
Schema unificado	db.go, migrations/	Pendente
Preimage encrypted	vault.go, engine.go	Pendente
PIN lockout	lockout.go, vault.go	Pendente

4. Phase 2: Adapters + Watchers (Semanas 3-4)

✓ **Objetivo:** Base funcional (adapters + watchers) para orchestrators consumirem dados reais.

4.1 Submarine Auto-Lock

Problema

Atualmente `CreateFromQuote()` deixa o swap em estado `open` sem `lockup_address`. O usuário precisa chamar `Lock()` manualmente.

Solução

```
// core/internal/swap/submarine.go
func (so *SubmarineOrchestrator) CreateFromQuote(ctx, quoteID) (*Swap, error) {
    // Validar quote
    quote, err := so.quotes.GetQuote(ctx, quoteID)
    if err != nil {
        return nil, err
    }

    // Criar swap
    swap, err := so.engine.Create(ctx, KindSubmarine, "regtest", 0)
    if err != nil {
        return nil, err
    }

    // AUTO-LOCK imediato
    swap, err = so.Lock(ctx, swap.ID, quoteID)
    if err != nil {
        return nil, err
    }

    return swap, nil
}
```

4.2 Reverse Swap Orchestrator

Arquivo: `core/internal/swap/reverse.go`

```
package swap

type ReverseOrchestrator struct {
    engine  *Engine
    db      *db.DB
    provider provider.Provider
    vault   *vault.Vault
}

func NewReverseOrchestrator(engine *Engine, database *db.DB, prov provider.Provider, v *vault.Vault) *ReverseOrchestrator {
    return &ReverseOrchestrator{
        engine:  engine,
        db:      database,
        provider: prov,
        vault:   v,
    }
}

func (ro *ReverseOrchestrator) CreateFromQuote(ctx, quoteID) (*Swap, error) {
    // 1. Gerar preimage R
    preimage := make([]byte, 32)
    rand.Read(preimage)
    hash := sha256.Sum256(preimage)

    // 2. Criar swap
    swap, err := ro.engine.Create(ctx, KindReverse, "regtest", 0)
    if err != nil {
        return nil, err
    }
}
```

```

// 3. Criptografar e salvar preimage
encPreimage, err := ro.vault.EncryptPreimage(preimage)
if err != nil {
    return nil, err
}

// 4. Atualizar DB
_, err = ro.db.ExecContext(ctx, `
    UPDATE swaps SET encrypted_preimage = ?, preimage_hash_hex = ?
    WHERE id = ?
`, encPreimage, hex.EncodeToString(hash[:]), swap.ID)

// 5. Lock com provider
return ro.Lock(ctx, swap.ID, quoteID)
}

func (ro *ReverseOrchestrator) PayInvoice(ctx, swapID) (*Swap, error) {
    // Pagar hold invoice do provider
    // Transição: Locked → waiting
}

func (ro *ReverseOrchestrator) Claim(ctx, swapID) (*Swap, error) {
    // Tentar MuSig2 key-path claim
    // Fallback: script-path claim
}

```

4.3 Chain Swap Orchestrator

Arquivo: core/internal/swap/chain.go

```

package swap

type ChainOrchestrator struct {
    engine *Engine
    db      *db.DB
    provider provider.Provider
}

func (co *ChainOrchestrator) CreateFromQuote(ctx, quoteID) (*Swap, error) {
    // Similar ao reverse: gerar R, criar swap, lock
}

func (co *ChainOrchestrator) FundBTC(ctx, swapID) (*Swap, error) {
    // Broadcast funding tx BTC
}

func (co *ChainOrchestrator) ClaimLiquid(ctx, swapID) (*Swap, error) {
    // Claim HTLC Liquid (revela R)
}

```

4.4 P2TR Verification

Arquivo: core/internal/swap/taproot.go

```

package swap

import (
    "github.com/btcsuite/btcd/txscript"
    "github.com/btcsuite/btcd/btcec/v2"
)

// VerifyP2TRAddress rebuilds o endereço P2TR localmente e compara
func VerifyP2TRAddress(
    claimPubkey, refundPubkey []byte,
    preimageHash []byte,
    timeoutBlocks int64,
    expectedAddress string,
) error {
    // 1. Construir swap tree
    tree, err := BuildSwapTree(KindSubmarine, SwapTreeParams{
        ClaimPubkey:  claimPubkey,
        RefundPubkey: refundPubkey,
        PreimageHash: preimageHash,
        TimeoutBlocks: timeoutBlocks,
    })
}

```

```

    if err != nil {
        return err
    }

    // 2. Gerar endereço P2TR
    address, err := tree.ToAddress(&chaincfg.ReggressionNetParams)
    if err != nil {
        return err
    }

    // 3. Comparar
    if address != expectedAddress {
        return fmt.Errorf("address mismatch: got %s, want %s", address, expectedAddress)
    }

    return nil
}

type SwapTree struct {
    InternalKey *btcec.PublicKey
    Scripts     [][]byte
}

func BuildSwapTree(kind Kind, params SwapTreeParams) (*SwapTree, error) {
    // Construir scripts conforme Boltz v2 spec
    // - Claim path: OP_SIZE 32 OP_EQUAL OP_IF OP_HASH256 OP_EQUALVERIFY OP_ELSE OP_CHECKLOCKTIMEVERI
}

```

4.5 MuSig2 Implementation

Arquivo: core/internal/musig2/session.go

```

package musig2

import (
    "github.com/btcsuite/btcd/btcec/v2"
    "github.com/btcsuite/btcd/btcec/v2/schnorr/musig2"
)

type Session struct {
    ID          string
    SwapID      string
    UserKey     *btcec.PublicKey
    ProviderKey *btcec.PublicKey
    AggKey      *musig2.AggregateKey
    SecNonce    [musig2.SecNonceSize]byte // CRÍTICO: nunca expor
    PubNonce    [musig2.PubNonceSize]byte
    AggNonce    *musig2.Nonces
    PartialSig  *musig2.PartialSignature
}

func NewSession(swapID string, userPriv *btcec.PrivateKey, providerPub *btcec.PublicKey) (*Session, error) {
    // Agregar chaves (ordem lexicográfica)
    keys := []*btcec.PublicKey{userPriv.PubKey(), providerPub}
    sort.Slice(keys, func(i, j int) bool {
        return bytes.Compare(keys[i].SerializeCompressed(), keys[j].SerializeCompressed()) < 0
    })

    aggKey, err := musig2.AggregateKeys(keys, false)
    if err != nil {
        return nil, err
    }

    return &Session{
        ID:          uuid.New().String(),
        SwapID:      swapID,
        UserKey:     userPriv.PubKey(),
        ProviderKey: providerPub,
        AggKey:      aggKey,
    }, nil
}

func (s *Session) GenerateNonce() (pubNonce []byte, err error) {
    // Gerar nonce com aux randomness
    var auxRand [32]byte
    rand.Read(auxRand[:])

    secNonce, pubNonce, err := musig2.GenNonces(

```

```

        musig2.WithCustomRand(bytes.NewReader(auxRand[:])),
    )
    if err != nil {
        return nil, err
    }

    s.SecNonce = *secNonce
    copy(s.PubNonce[:], pubNonce[:])

    return s.PubNonce[:], nil
}

func (s *Session) AggregateNonces(providerNonce []byte) error {
    nonces := [][]musig2.PubNonceSize{byte{s.PubNonce}}
    var provNonce [musig2.PubNonceSize]byte
    copy(provNonce[:], providerNonce)
    nonces = append(nonces, provNonce)

    aggNonce, err := musig2.AggregateNonces(nonces)
    if err != nil {
        return err
    }

    s.AggNonce = aggNonce
    return nil
}

func (s *Session) Sign(txHash []byte, userPriv *btcec.PrivateKey) (partialSig []byte, err error) {
    var msg [32]byte
    copy(msg[:], txHash)

    partialSig, err = musig2.Sign(
        &s.SecNonce,
        userPriv,
        s.AggNonce,
        s.AggKey.FinalKey,
        msg,
    )
    if err != nil {
        return nil, err
    }

    // Cleanup secononce
    for i := range s.SecNonce {
        s.SecNonce[i] = 0
    }

    return partialSig, nil
}

func (s *Session) Cleanup() {
    // Zero sensitive data
    for i := range s.SecNonce {
        s.SecNonce[i] = 0
    }
}

```

4.6 Watcher + WebSocket Integration

Arquivo: core/internal/watcher/watcher.go

```

// Adicionar handler para WS updates
func (w *Watcher) HandleWSUpdate(swapID, boltzStatus string, tx *boltz.TxInfo) {
    // 1. Buscar swap
    swap, err := w.engine.Get(context.Background(), swapID)
    if err != nil {
        log.Printf("Failed to get swap %s: %v", swapID, err)
        return
    }

    // 2. Normalizar status usando boltz.Normalize existente
    mapping := boltz.Normalize(boltzStatus, swap.Kind)

    // 3. Se requer ação, reconciliar
    if mapping.Action != boltz.ActionNone {
        go w.reconcileSwap(context.Background(), swap)
    }
}

```

```
}  
}
```

4.7 Checklist Phase 2

Tarefa	Arquivos	Status
Auto-Lock	submarine.go	Pendente
Reverse Swap	reverse.go	Pendente
Chain Swap	chain.go	Pendente
P2TR Verification	taproot.go	Pendente
MuSig2	musig2/session.go	Pendente
Watcher+WS	watcher.go	Pendente

5. Phases 3-6: Resumo

5.1 Phase 3: Swap Protocol (Semanas 5-6)

Entregas

- **Submarine Auto-Lock + Claim/Refund** - MuSig2 + script-path
- **Reverse Swap** - `core/internal/swap/reverse.go`
- **P2TR Verification** - `core/internal/swap/taproot.go`
- **MuSig2** - `core/internal/musig2/session.go`
- **Reconcile + Invariantes** - Determinístico

5.2 Phase 4: Electron Migration (Semanas 7-8)

Entregas

- **Electron Main** - `electron/main.ts` com hardening
- **IPC Bridge** - `electron/preload.ts` com whitelist + Zod
- **Frontend Migration** - Alterar `api.ts` para IPC
- **Deprecar api-bridge** - Manter apenas para debug

5.3 Phase 5: Tx Management (Semana 9)

Entregas

- **RBF/CPFP** - `core/internal/tx/bumper.go`
- Integração com LND WalletKit + elementsd

5.4 Phase 6: Packaging (Semana 10)

Entregas

- **Windows Installer** - Inno Setup
- **macOS DMG** - Notarização Apple
- **Code Signing** - Authenticode + Developer ID
- **E2E Tests** - Submarine, Reverse, Chain, Crash Recovery

6. Guia de Desenvolvimento

6.1 Setup do Ambiente

Pré-requisitos

```
# Go 1.21+  
go version  
  
# Node.js 18+  
node --version  
  
# Docker (para regtest)  
docker --version
```

Clone e Build

```
# Clone  
git clone https://github.com/4LFR3Dv1/XSWallet.git  
cd XSWallet  
  
# Build Go Core  
cd core  
go build ./cmd/xscore  
./xscore.exe --network=regtest --port=9735  
  
# Frontend (dev)  
cd ../frontend  
npm install  
npm run dev  
  
# API Bridge (temporário)  
cd ../api-bridge  
npm install  
npm start
```

6.2 Regtest Environment

```
# Subir regtest  
docker compose -f test/regtest/docker-compose.yml up -d  
  
# Verificar  
docker compose -f test/regtest/docker-compose.yml ps
```

6.3 Workflow de Desenvolvimento



6.4 Convenções de Código

Go

- Seguir `gofmt`
- Comentários em português ou inglês (consistente por arquivo)
- Testes em `*_test.go`

TypeScript/React

- ESLint + Prettier

- Componentes funcionais com hooks
- Types explícitos (evitar `any`)

6.5 Testes

Unit Tests

```
# Go
cd core
go test ./...

# Frontend
cd frontend
npm test
```

Integration Tests

```
# Regtest
cd test/e2e
go test -tags=integration ./...
```

7. Referências

7.1 Documentação

Documento	Caminho	Descrição
Especificação Técnica	docs/XS_Wallet_Especificacao_Tecnica_v2.html	Fonte de verdade (humano)
Status vs Spec	docs/STATUS_ESPECIFICACAO_XS_WALLET.md	Gap analysis
Plano de Implementação	docs/PLANO_IMPLEMENTACAO_v0.2.md	Roadmap detalhado
Agent Guide	docs/AGENTS.md	Guia para agentes

7.2 Arquitetura

Componente	Caminho
Go Core	core/
Swap Engine	core/internal/swap/
Vault	core/internal/vault/
Boltz Client	core/internal/boltz/
Database	core/internal/db/
Frontend	frontend/
Protos	proto/

7.3 Recursos Externos

- Boltz API v2: api.docs.boltz.exchange
- BIP327 (MuSig2): github.com/bitcoin/bips
- LND gRPC: lightning.engineering/api-docs
- Elements RPC: elementsproject.org/en/doc

XS Wallet - Guia de Implementação v1.0.0

Janeiro 2026 • Baseado em Especificação Técnica v0.2.0

Self-Custody • Zero Trust • Don't Trust, Verify